

Nome: Eduardo Lucas Lemes Januário CT3037568

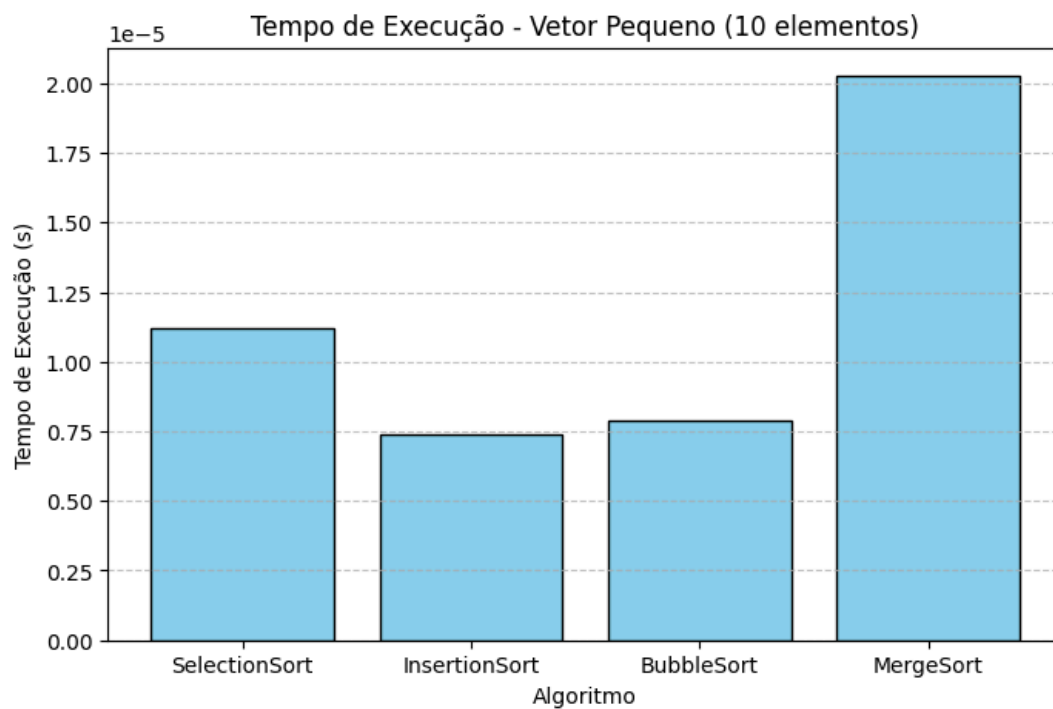
Nome: Guilherme Batista CT3037274

Prática Algoritmos de Ordenação

<https://colab.research.google.com/drive/1EHLua1VqNCvfvNvFFPAc6sICVJJWZhwC?usp=sharing>

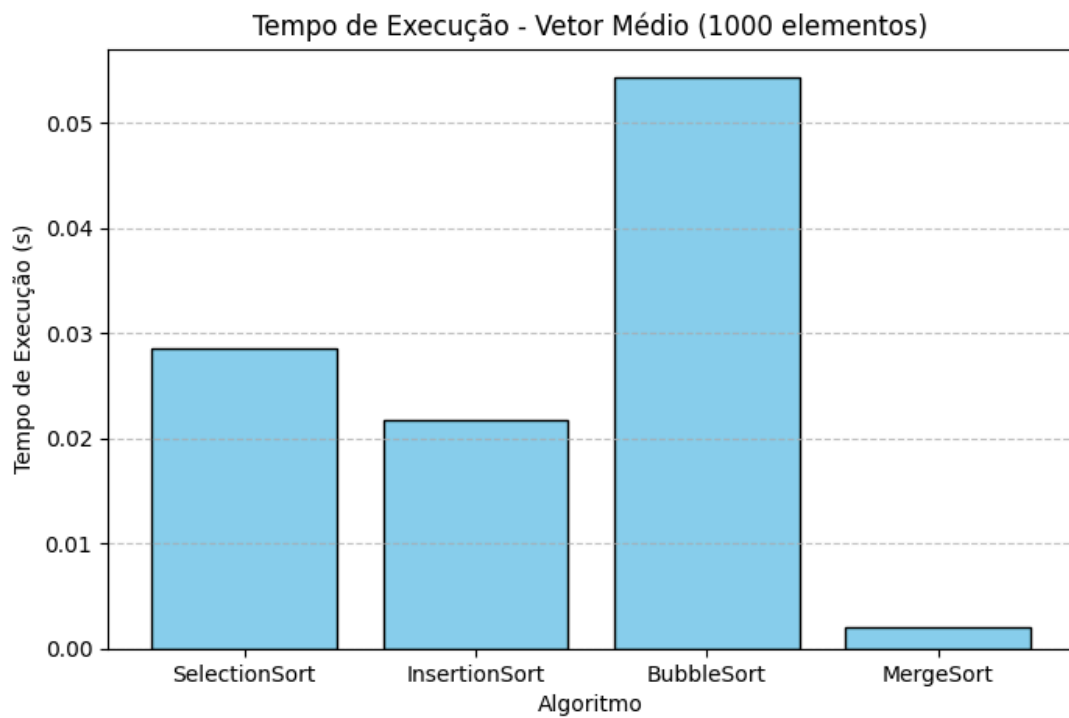
Testes no Vetor Pequeno

Os 10 primeiros números ordenados	
[4, 9, 10, 11, 17, 23, 28, 60, 76, 93]	
Algoritmo	Tempo de Execução (s)
SelectionSort	0.000011
InsertionSort	0.000007
BubbleSort	0.000008
MergeSort	0.000020



Testes no Vetor Médio

Os 10 primeiros números ordenados	
[10, 23, 24, 33, 52, 56, 58, 59, 68, 76]	
Algoritmo	Tempo de Execução (s)
SelectionSort	0.028484
InsertionSort	0.021722
BubbleSort	0.054304
MergeSort	0.001988



Relatório

1. Qual algoritmo foi mais rápido no vetor pequeno?

O Algoritmo InsertSort, em razão do custo baixo de comparação e movimentação.

2. Qual foi mais rápido no vetor médio?

O Algoritmo MergeSort, em razão de sua complexidade $O(n \log n)$ que garante um crescimento muito mais controlado do tempo de execução em comparação aos algoritmos quadráticos.

3. Os algoritmos $O(n^2)$ apresentaram diferenças significativas entre si?

No vetor pequeno os algoritmos ficaram praticamente empatados e não mostraram muita diferença entre si, somente o selection sort que demorou um pouco a mais que os outros, tal diferença pode ser explicada pelo fato dele percorrer toda a lista à procura do menor elemento, mesmo que a lista já esteja quase ordenada. Entretanto, no vetor médio as diferenças se tornam mais evidentes, por exemplo o bubble sort que mostrou uma diferença negativa em comparação aos outros algoritmos, devido ao grande número de trocas realizadas a cada passagem.

4. O Merge Sort foi vantajoso em quais casos?

Em grande quantidade de dados, por exemplo no vetor médio que tem muitos números, o merge sort se mostrou muito eficaz, fez sua ordenação em um tempo muito menor aos demais, já que os algoritmos quadráticos escalam muito mal conforme o tamanho do vetor cresce.